Basic (plug search in as a plain tool first) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The exact pieces:

- `@tool` on `def search_docs(query: str) -> str:` — its docstring is what the model reads to decide when to call it, so say *when* to use it.
- Inside: `retrieve(collection, query)`, then collect each `result["text"]` in a list and `"\n\n".join(...)` it.
- `model.bind_tools([search_docs])` in `think`; in `act`, `search_docs.invoke(call["args"])` for every tool call, sent back as a `{"role": "tool", ...}` message (Doc07).

Then the real-project questions: what should the tool return when nothing is found (an explicit sentence, not an empty string)? What if `retrieve()` raises (catch it and return a sentence the model can read)? How many chunks is too many (pass `k=`)? And if a later approval step needs to show *which files* were used, keep the sources separately from the joined text.

Hint 2 — The plan, and almost the whole thing

```
search_docs(query: str) -> str:
    try: results = retrieve(collection, query, k=MAX_RESULTS)
    except Exception as exc: return "Search is temporarily unavailable: ..."
    if not results: return "No relevant documents found."
    join the texts

@dataclass SearchResult: text: str, sources: list[str]
search_docs_with_sources(query) -> SearchResult   (same logic, keeps
                                                   each source_file)
```

One piece to get you unstuck:

```
def join_texts(results: list[dict]) -> str:
    pieces: list[str] = []
    for result in results:
        pieces.append(result["text"])
    return "\n\n".join(pieces)
```

Full solution: Show me the solution